AGENTIC AI ARCHITECTURE

PrestoDB + EKS Auto-Scaling
Replacing Selenium with Autonomous Agents

Financial-grade large query execution on AWS EKS · 0 → 80 pods · fully agentic

CURRENT APPROACH

Selenium Automation

Script-driven, brittle UI automation. Reactive scaling logic. Manual intervention when thresholds or UI change. Limited observability & retry logic.

→

AGENTIC AI APPROACH

Autonomous Agent Loop

LLM-orchestrated agent with tools for query analysis, scaling decisions, EKS control, monitoring, and self-healing. Adapts in real-time.

— AGENT EXECUTION FLOW —

STEP 01 · PERCEPTION

Query Ingestion & Complexity Analysis

The agent receives the large financial query. It uses an LLM tool call to analyze estimated scan volume, join complexity, and memory footprint — producing a scaling recommendation before execution begins.

`analyze_query(sql)` `estimate_resource_need()` `Presto EXPLAIN plan`

STEP 02 · PLANNING

Dynamic Scaling Plan

Based on query profile, the agent reasons over current EKS cluster state (node count, CPU/mem headroom) and decides the scaling target — e.g. 0 → 80 pods — with staged ramp-up milestones and checkpoints.

get_cluster_state() plan_scaling(target=80) set_milestones()

STEP 03 · ACTION

EKS Scale-Out Execution

Agent calls Kubernetes API tools to apply HPA/Karpenter scaling rules, patch the PrestoDB worker Deployment, and trigger node provisioning. It monitors readiness gates — only proceeding when pods are Running + Ready.

kubectl_patch_deployment() apply_hpa_config() karpenter_provision() watch_pod_readiness()

STEP 04 · EXECUTION

Query Dispatch & Live Monitoring

Agent submits the query to PrestoDB coordinator, tracks query ID, polls stage-level progress, and streams metrics to an observability tool. It detects spill-to-disk events, GC pressure, or straggler tasks autonomously.

presto_submit_query(sql) poll_query_progress(id) stream_metrics() detect_anomalies()

STEP 05 · SELF-HEALING

Adaptive Remediation Loop

If the agent detects a stalled stage, OOM worker, or query failure, it autonomously decides: retry with adjusted memory limits, reschedule on fresh nodes, or split the query into sub-queries. No human in the loop required.

retry_with_params() split_query() replace_failed_pod() adjust_memory_config()

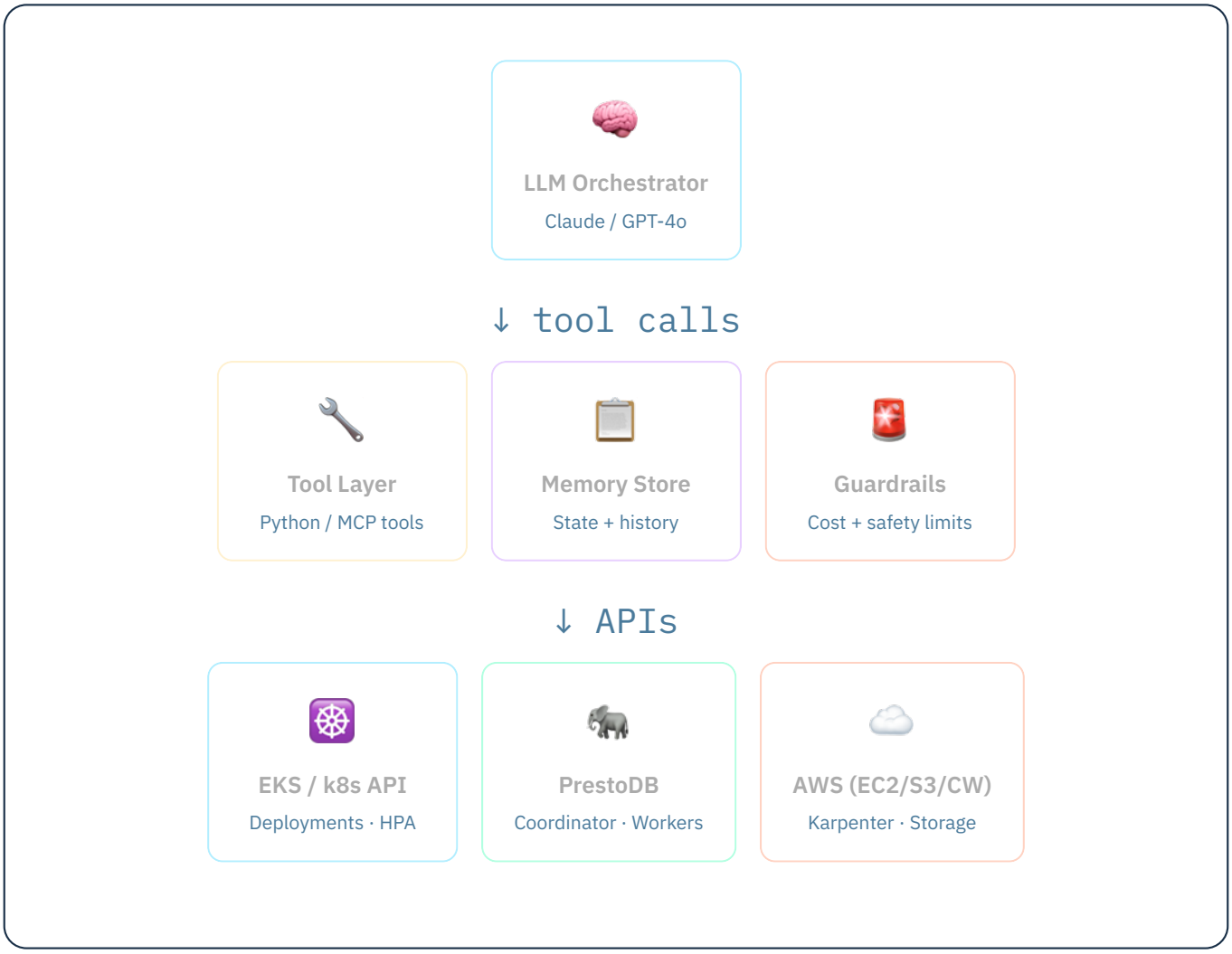
STEP 06 · TEARDOWN

Results Delivery & Scale-In

On success, agent exports results to S3/Redshift, generates an execution report (cost, duration, pod-hours), and triggers scale-in — returning the cluster to zero workers. Audit trail written to your logging system.

`export_results_s3()` `generate_report()` `scale_in_cluster()` `write_audit_log()`

— COMPONENT ARCHITECTURE —



— WHY AGENTIC VS. SELENIUM —

🎯

Intent-Driven

Agent understands the goal, not just the steps. It adapts when the environment changes — no brittle UI selectors.

🔁

Self-Healing

Detects failures mid-flight and autonomously remediates — retry, re-route, or escalate — without waking an engineer.

💰

Cost-Aware

Guardrails enforce pod-hour budgets and trigger early scale-in. Selenium has no native cost reasoning.

📈

Elastic by Design

Each query gets a fresh scaling plan. Handles 0→80 or 0→200 without code changes — LLM reasons over state.

📝

Full Audit Trail

Every tool call, decision, and metric is logged. Meets financial-grade compliance requirements out of the box.

🔌

MCP-Ready

Tools are exposed as MCP servers — composable, testable, and reusable across agents and workloads.